

DNS Behaviour Monitor

Analysing Normal and Unusual DNS Patterns in Network Traffic

By

SAJAN TAMANG

Submitted to

The University of Roehampton

In partial fulfilment of the requirements

for the degree of

BACHELOR OF SCIENCE IN CYBER SECURITY

Supervisor: Badis Aoun

Student ID: TAM22614816

May 2026

Declaration

I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

Signed: 

Name: Sajan Tamang

Date: 06/05/2026

Acknowledgements

The author would like to thank Badis Aoun, project supervisor at the University of Roehampton, for guidance, formative feedback on the project poster, and direction during the midpoint review. Particular thanks are due for the steer towards a behavioural rather than security-detection framing of the project, which shaped the analytical scope of the work.

The author also acknowledges the wider University of Roehampton Computer Science teaching team for the foundational networking and cyber security modules that underpin the technical work, and fellow students whose informal feedback on early dashboard prototypes helped sharpen the user-facing observations.

Finally, the author thanks family and friends for their support throughout the final year of study.

Abstract

The Domain Name System (DNS) underpins virtually every interaction on the modern Internet, yet in small networks DNS traffic is rarely examined beyond basic operational counts. Existing tooling either filters traffic, provides simple totals, or operates at Internet scale with security-focused detection in mind, leaving a gap for lightweight, transparent behavioural analysis in controlled environments.

This project addresses that gap by designing and implementing the DNS Behaviour Monitor: a containerised data-collection environment built around Pi-hole running in Docker, a Python pipeline that parses dnsmasq logs into a normalised schema, computes baseline behavioural metrics, applies four transparent heuristics for unusual behaviour, and presents the results through a Streamlit dashboard.

The final artefact was evaluated using controlled baseline, burst, NXDOMAIN, and long-domain datasets, supported by an additional sanity-check dataset and dashboard evidence screenshots. The parser achieved a 100 percent parse-success rate against query lines in the evidence screenshots, exceeding the 95 percent target. The baseline dataset remained clear; the burst dataset triggered burst and rare-TLD observations; the long-domain and NXDOMAIN datasets triggered the expected long-domain and NXDOMAIN-excess observations. The sanity-check dataset further demonstrated how the dashboard highlights long-domain behaviour in a realistic mixed log.

The project contributes a reproducible behavioural analysis pipeline suitable for educational use. It remains deliberately analytical rather than threat-focused, with outputs presented as observations that require contextual interpretation rather than as security detections.

Table of Contents

Declaration.....	2
Acknowledgements.....	3
Abstract.....	4
List of Figures	7
List of Tables	7
1. Introduction	8
1.1 Problem Statement and Research Question.....	8
1.2 Aims.....	9
1.3 Objectives.....	9
1.4 Scope.....	9
2. Background and Context.....	10
2.1 Background	10
2.2 Literature Review	10
2.2.1 Foundational DNS architecture and behaviour	10
2.2.2 DNS performance and traffic characteristics	10
2.2.3 Large-scale DNS measurement	11
2.2.4 Practical DNS logging and tooling	11
2.2.5 Synthesis and implications for this project	11
2.3 Competitor Analysis and Related Work	12
3. Tools and Methods	14
3.1 Rationale for Technical Tools Selected	14
3.1.1 DNS log collection	14
3.1.2 Programming language and data processing.....	14
3.1.3 Visualisation and dashboard framework	14
3.1.4 Data storage.....	14
3.2 Rationale for Research Methods Used	15
3.2.1 Exploratory data analysis	15
3.2.2 Baseline-driven behaviour analysis.....	15
3.2.3 Heuristic identification of unusual behaviour.....	15
3.3 Project Management and Product Lifecycle.....	15
3.4 Risk Register	16
3.5 Legal, Ethical, and Professional Issues	17
3.5.1 Legal considerations.....	17
3.5.2 Ethical considerations	17
3.5.3 Professional considerations	17
4. Requirements and Specifications.....	18

4.1 Intended Users and Personas	18
4.2 User Stories	18
4.3 User Journey	18
4.4 Use Case Descriptions	19
4.5 Functional Requirements	20
4.6 Non-Functional Requirements	20
5. Solution Design	21
5.1 High-Level Architecture	21
5.2 Data Flow and Processing Pipeline	22
5.3 Data Model and Derived Features	22
5.4 Core Modules and Responsibilities	23
5.5 Heuristic Design	25
5.6 Design Constraints and Assumptions	25
6. Solution Implementation	26
6.1 Implementation Overview	26
6.2 Sprint 1: Containerised Data Source	26
6.3 Sprint 2: Traffic Generation and Dataset Export	26
6.4 Sprint 3: Log Parser and Schema	26
6.5 Sprint 4: Feature Extraction and Metrics	27
6.6 Sprint 5: Heuristic Engine	27
6.7 Sprint 6: Streamlit Dashboard	27
6.8 Sprint 7: Automation and Test Harness	28
7. Solution Testing and Evaluation	29
7.1 Evaluation Objectives	29
7.2 Methodology	29
7.3 Technical Evaluation: Parser and Pipeline	29
7.4 Dataset-Based Behavioural Evaluation	30
7.5 Dashboard Usability Evaluation	31
7.6 Evaluation Metrics Summary	31
7.7 Limitations of the Evaluation	31
8. Discussion	32
9. Conclusion	33
9.1 Summary	33
9.2 Recommendations for Further Work	33
9.3 Reflection	33
10. References	35
11. Appendices	36
Appendix A — Project Proposal Poster	36

Appendix B — Ethics Form..... 37

Appendix C — GitHub Repository and Project Management..... 40

Appendix D — Risk Register..... 41

Appendix E — Supervisor Communications and Mid-point Review Feedback..... 42

Appendix F — Use of Generative AI..... 42

Appendix G — Testing and Dashboard Evidence..... 43

List of Figures

- Figure 1. User flow diagram showing the intended interaction sequence.
- Figure 2. Use case diagram for the DNS Behaviour Monitor.
- Figure 3. High-level system architecture of the DNS Behaviour Monitor.
- Figure 4. Data flow diagram showing the DNS behaviour analysis pipeline.
- Figure 5. Entity relationship diagram for the logical data model.
- Figure 6. UML class diagram showing the main software components.
- Figure 7. UML sequence diagram for the DNS log analysis workflow.
- Figure 8. Streamlit dashboard KPI and heuristic-status view.
- Figure 9. Project proposal poster.
- Figure 10. Ethics form evidence, page 1.
- Figure 11. Ethics form evidence, page 2.
- Figure 12. Ethics form evidence, page 3.
- Figure 13. GitHub repository and project management evidence.
- Figure 14. Baseline dashboard evidence.
- Figure 15. Sanity-check dashboard evidence.
- Figure 16. Burst dashboard evidence.
- Figure 17. Long-domain dashboard evidence.
- Figure 18. NXDOMAIN dashboard evidence.

List of Tables

- Table 1. Comparative feature analysis of existing DNS monitoring tools and the proposed system.
- Table 2. Project risk register and mitigation status.
- Table 3. Heuristic rules and threshold logic.
- Table 4. Parse-success evidence from dashboard screenshots.
- Table 5. Heuristic observations across evaluation datasets.
- Table 6. Evaluation metrics, methods, and outcomes.

1. Introduction

The Domain Name System (DNS) is a fundamental component of Internet communication, enabling devices and applications to access networked resources by translating human-readable domain names into numerical Internet Protocol (IP) addresses. Every interaction with web services, cloud platforms, and many background system processes depends on DNS resolution as an initial step. As a result, DNS queries are generated continuously, producing a detailed and persistent stream of network metadata.

Despite its central role, DNS is typically treated as a transparent infrastructure service that is only examined when failures occur or when security incidents are suspected. In small-scale, educational, or laboratory environments DNS traffic is rarely analysed beyond basic operational metrics such as total query counts or blocked domains. This limited examination reduces opportunities to understand how systems normally interact with DNS and how user activity, application behaviour, or configuration changes shape observable DNS usage patterns.

This project positions DNS traffic as a behavioural data source rather than a purely operational or security artefact. By analysing DNS logs collected within a controlled environment, the project characterises normal DNS behaviour and highlights unusual patterns without assuming malicious intent. The emphasis is on understanding variation and deviation as an analytical and educational exercise rather than developing a detection or enforcement system.

1.1 Problem Statement and Research Question

DNS traffic generally follows predictable behavioural patterns shaped by user activity, application behaviour, caching mechanisms, and resolver configuration. Deviations from these patterns can arise from benign causes including software updates, automation, experimental applications, or misconfigurations. In many cases, such deviations are either ignored or framed immediately through a security lens.

Existing DNS monitoring tools and research outputs primarily target enterprise-scale environments or security detection use cases. They often prioritise alerting, classification, or threat identification, and typically assume access to large datasets and specialist infrastructure. As a result they are poorly suited to small or controlled environments where the goal is exploratory understanding rather than detection or enforcement.

The problem addressed by this project is the lack of lightweight, transparent approaches for analysing DNS behaviour in controlled environments that focus on behavioural understanding rather than security classification. There is a need for a system that enables users to observe, visualise, and interpret DNS behaviour patterns, and to identify unusual activity relative to established baselines without attributing intent.

Research Question: How can DNS behaviour be characterised and visualised in a controlled environment in order to distinguish normal usage patterns from unusual DNS activity without relying on security-focused detection techniques?

1.2 Aims

The overarching aim of this project is to improve understanding of DNS behaviour by analysing DNS traffic patterns within a controlled network environment. Three subordinate aims structure the work:

- Analyse DNS traffic to characterise normal behavioural patterns, including temporal, structural, and distributional characteristics.
- Identify and highlight unusual DNS behaviours by comparing observed activity against established behavioural baselines, without assigning malicious intent.
- Design and implement a lightweight DNS behaviour monitoring tool that transforms raw DNS logs into interpretable visual representations to support behavioural insight and learning.

1.3 Objectives

The aims are operationalised through the following objectives:

- Collect DNS query logs from a controlled local environment using lightweight DNS logging tools (Pi-hole and the underlying dnsmasq resolver).
- Pre-process and normalise raw DNS logs into a consistent schema covering timestamp, queried domain, query type, client identifier, response code, and source.
- Establish baseline DNS behaviour metrics, including query volume over time, response code distributions, top-level domain (TLD) usage, and domain name structural characteristics.
- Define transparent heuristic criteria for identifying unusual DNS behaviour, including repeated failed lookups, excessive domain length, uncommon TLDs, and abnormal query bursts.
- Develop a Python-based DNS behaviour analysis engine that computes behavioural metrics and highlights deviations relative to established baselines.
- Implement a lightweight Streamlit dashboard that visualises DNS behaviour trends and highlighted observations in an interpretable manner.
- Plan and conduct an evaluation of the system using multiple DNS datasets generated under varied but controlled conditions.

1.4 Scope

The project explicitly focuses on DNS behaviour analysis rather than security detection or threat classification. While unusual DNS patterns may resemble indicators discussed in security literature, the system does not attempt to label activity as malicious. All observations are presented as behavioural deviations that require contextual interpretation. The scope is deliberately limited to controlled environments in order to support transparency, reproducibility, and ethical data handling.

2. Background and Context

2.1 Background

The Domain Name System is a hierarchical, distributed naming service introduced to address the scaling limitations of static host tables. Every DNS resolution involves a chain of caching, recursion, and delegation steps, and the resulting traffic carries useful structural metadata: the timestamp of the request, the queried domain, the type of record sought, and the response code returned by the resolver.

DNS behaviour is shaped by several interacting factors: resolver design, caching parameters, application polling, background system processes, and operating-system service discovery. Caching, in particular, introduces natural variability into DNS traffic. Periodic bursts and repeated queries are commonly observed when cache entries expire, with no malicious origin. Deviations from expected DNS patterns are therefore not inherently abnormal in a security sense — they may simply reflect software updates, service discovery, or configuration changes. Understanding these baseline behaviours is essential before any meaningful interpretation of unusual DNS activity can be attempted.

This project adopts a behavioural perspective on DNS analysis by treating DNS logs as an empirical data source rather than as indicators of faults or threats. By analysing DNS logs collected within a controlled environment, the project characterises normal DNS behaviour patterns and highlights deviations without attributing intent. This contextual and exploratory approach underpins the analytical methods and system design choices described in later sections.

2.2 Literature Review

2.2.1 Foundational DNS architecture and behaviour

Mockapetris [1] introduced the Domain Name System as a distributed and hierarchical naming architecture designed to provide scalability, resilience, and decentralised administration. The associated specification [2] defines DNS message formats, response codes (NOERROR, NXDOMAIN, SERVFAIL, REFUSED), and resolver behaviour, providing the conceptual baseline for what constitutes correct DNS operation. Both documents are specification-oriented and do not consider empirical traffic analysis or behavioural variation. The current project builds upon these architectural principles by translating DNS protocol semantics into measurable behavioural metrics, enabling DNS logs to be analysed as behavioural data rather than purely as protocol events.

2.2.2 DNS performance and traffic characteristics

Jung et al. [3] presented one of the earliest large-scale empirical analyses of DNS traffic and demonstrated the significant impact of caching on query frequency, latency, and burst behaviour. The authors showed that repeated queries and sudden spikes are often legitimate outcomes of cache expiry rather than abnormal activity. This finding directly

motivates the present project's decision to adopt baseline-driven thresholds rather than fixed cut-offs for the burst-window heuristic. Crovella and Krishnamurthy [4] complement this work with a methodological treatment of Internet measurement, including baseline construction, heavy-tailed distributions, and the risk of overinterpreting outliers. Although their text is not DNS-specific, the principles of measurement bias and contextual interpretation are applied directly in the present design.

2.2.3 Large-scale DNS measurement

DNS-OARC [5] publishes authoritative DNS measurement reports based on large-scale operational datasets, demonstrating that DNS traffic exhibits consistent baseline characteristics even at Internet scale. Cloudflare research [6] reinforces this view by presenting traffic analytics drawn from one of the world's largest public resolvers. Both sources, however, target professional operators and assume access to specialist infrastructure. The present project draws conceptual inspiration from their baseline-driven framing while adapting it to a lightweight, transparent system suitable for controlled and educational environments.

2.2.4 Practical DNS logging and tooling

Pi-hole [7] is a widely deployed DNS sinkhole and logger commonly used in home networks and small laboratories. Its documentation establishes that DNS logs can be collected efficiently and at low cost. Pi-hole's analytics, however, are limited to predefined operational metrics — total queries, top domains, blocked counts — and do not include behavioural baseline modelling or contextual interpretation of unusual patterns. The present project uses Pi-hole purely as a log source and treats the underlying dnsmasq logs as raw behavioural data on which custom analysis is performed.

2.2.5 Synthesis and implications for this project

Across the reviewed literature DNS is consistently treated as critical infrastructure, yet its behavioural characteristics are most often secondary to concerns of correctness, performance, or security. Foundational specifications define how DNS should operate but not how it varies in practice. Performance-focused studies provide insight into caching and burstiness but frame behaviour primarily through the lens of optimisation. Large-scale measurement initiatives demonstrate that baselines exist, but their methods presuppose datasets and infrastructure not available in small networks. Practical tools such as Pi-hole provide accessible logging but limited analytical depth.

A common limitation across much of this work is the tendency to interpret deviations through a security or fault-detection perspective. The present project adopts a different stance, treating unusual DNS behaviour as an observation that requires contextual explanation rather than immediate classification. This shift is directly motivated by the literature, which demonstrates that DNS behaviour is shaped by multiple benign factors. The findings of the literature review have therefore guided three concrete design choices in this project: protocol-faithful parsing of dnsmasq logs grounded in [1], [2]; baseline-driven heuristics rather than fixed thresholds, motivated by [3] and [4]; and a lightweight, transparent dashboard inspired conceptually by [5] and [6] but appropriate for the small, controlled scale described in [7].

2.3 Competitor Analysis and Related Work

This subsection compares the DNS Behaviour Monitor with four related artefacts encountered during the literature search and tool evaluation.

Pi-hole. Pi-hole functions as a DNS sinkhole that intercepts DNS queries and blocks requests to known advertising and tracking domains. All DNS queries passing through the resolver are logged and summarised through a web-based dashboard. Pi-hole is open-source and free to use, with a low hardware footprint. Its analytics, however, are restricted to operational summaries — total queries, top domains, blocked requests — without baseline modelling or contextual interpretation of unusual behaviour. In this project Pi-hole is used as a DNS log collection mechanism rather than as an analytical solution. The proposed system extends Pi-hole's functionality by transforming its logs into behavioural metrics, establishing baselines, and highlighting deviations with explanatory context.

dnsmasq. dnsmasq is a lightweight DNS forwarder and DHCP server commonly deployed on routers and embedded systems. It produces detailed DNS logs but provides no built-in analytics or visualisation; the analytical burden falls entirely on external tooling. In the present project dnsmasq is the underlying logger inside the Pi-hole container, and the parser is written against its log format directly. This means the parser would equally accept a dnsmasq deployment without Pi-hole, broadening the system's applicability.

Cloudflare DNS Analytics. Cloudflare provides DNS analytics for traffic processed through its global resolver infrastructure, with high-quality visualisation, massive datasets, and strong performance metrics. Access to detailed analytics is tightly coupled to Cloudflare's managed services and is unsuitable for small or offline environments. Cloudflare's framing also emphasises performance and security rather than behavioural exploration. The present system adopts similar baseline-driven concepts but prioritises transparency, reproducibility, and small-scale applicability.

DNS-OARC. DNS-OARC is a research consortium that produces measurement datasets, tools, and reports to support DNS reliability research at Internet scale. Its outputs are highly authoritative but assume specialist knowledge and infrastructure, and are not interactive or exploratory. The present system adapts DNS-OARC's baseline-driven philosophy into a lightweight, hands-on tool suitable for controlled environments and learning contexts.

The comparative position of the proposed system against these tools is summarised in Table 1.

Table 1. Comparative feature analysis of existing DNS monitoring tools and the proposed system.

Feature	Pi-hole	dnsmasq	Cloudflare	DNS-OARC	Proposed system
Open source	Yes	Yes	No	Partial	Yes
Cost	Free	Free	Paid	Membership	Free
Behavioural baselines	No	No	Limited	Yes	Yes
Unusual-behaviour highlighting	No	No	Limited	Yes	Yes
Visualisation	Basic	None	Advanced	Limited	Interpretable
Operating scale	Small	Small	Enterprise	Internet	Small / controlled
Methodological transparency	Medium	High	Low	Medium	High
Educational suitability	Medium	Low	Low	Low	High

The comparison shows that existing tools prioritise either operational filtering, raw logging, enterprise-scale analytics, or research-grade measurement. None are explicitly designed for exploratory DNS behaviour analysis in controlled or educational environments using lightweight, transparent methods. The proposed DNS Behaviour Monitor fills this gap by combining accessible DNS log sources with baseline-driven analysis and interpretable visualisation.

3. Tools and Methods

3.1 Rationale for Technical Tools Selected

3.1.1 DNS log collection

DNS logs are collected from Pi-hole running inside a Docker container on the local development machine. Pi-hole exposes its DNS service on host port 5354 and its admin interface on host port 8081, leaving the standard host port 53 untouched and avoiding conflicts with the Windows DNS Client service. Pi-hole writes dnsmasq-format logs to a bind-mounted directory on the host, which the export script copies into the project's data folder. Pi-hole was selected over a bare dnsmasq deployment because it bundles a known, stable dnsmasq version with predictable logging behaviour, accelerates reproducibility through Docker, and is the most common entry point to dnsmasq logs in the home-network and educational contexts that motivate this project. Enterprise platforms such as Infoblox and managed cloud resolvers were considered but rejected as opaque, costly, and unsuitable for transparent academic analysis.

3.1.2 Programming language and data processing

The analysis engine is implemented in Python 3.10+. Python was chosen for its strong support for parsing and tabular data manipulation through pandas, its readability for review purposes, and its ecosystem alignment with networking and Internet measurement research. Lower-level alternatives such as C and C++ were rejected as adding development complexity without justification at the data volumes considered. Java was considered but offered no clear advantage for an exploratory pipeline.

3.1.3 Visualisation and dashboard framework

The dashboard is implemented with Streamlit and Plotly. Streamlit allows rapid construction of data-driven interfaces directly from Python, making it well suited to a single-developer analytical project. Plotly was selected for its interactive charting, native theming, and the breadth of chart types required (time-series, categorical bars, donuts, heatmaps). Front-end frameworks such as React were considered but rejected as disproportionate to the project's analytical goals.

3.1.4 Data storage

Datasets and derived metrics are stored as plain text logs and CSV/JSON outputs on the local filesystem. This approach aligns with data-minimisation principles, supports reproducibility, and avoids any cloud transmission of network metadata. SQLite was considered as a structured store but rejected as unnecessary at the project's data scale.

3.2 Rationale for Research Methods Used

3.2.1 Exploratory data analysis

The project adopts Exploratory Data Analysis (EDA) as its primary research methodology. EDA is appropriate when system behaviour is influenced by multiple interacting factors that are not fully understood in advance, and when the analyst seeks to characterise data shape rather than confirm a pre-specified hypothesis. EDA allows the project to avoid the strong assumptions of malicious intent that pervade existing DNS-monitoring literature and to remain faithful to the behavioural framing of the research question.

3.2.2 Baseline-driven behaviour analysis

A baseline-driven approach is used to characterise normal DNS behaviour before identifying deviations. Baseline metrics include query volume over time, response code distribution, TLD usage, domain length, and label-count statistics. Baselines provide essential context for interpreting deviations and reduce the risk of overinterpreting normal variability — a risk explicitly highlighted by Jung et al. [3] and Crovella and Krishnamurthy [4].

3.2.3 Heuristic identification of unusual behaviour

Unusual DNS behaviour is identified using transparent, rule-based heuristics. Each heuristic is documented in code, exposes its threshold logic in plain Python, and produces an evidence record explaining why a finding was raised. Machine-learning approaches such as autoencoders and isolation forests were considered but rejected on three grounds: a lack of labelled training data appropriate to the project scope, reduced explainability for marking and supervisor review, and a misalignment with the project's analytical and educational goals.

3.3 Project Management and Product Lifecycle

The project followed an iterative, sprint-based lifecycle informed by lightweight agile practices. Work was tracked using a GitHub Projects Kanban board with four lanes (planned, ready, in progress, in review) and weekly cadence reviews aligned to supervisor meetings. The Git repository hosted on GitHub provided version control, with commits structured to reflect each functional milestone — log parsing, feature extraction, metrics, heuristics, dashboard, and evaluation harness. An early proof-of-concept (delivered for the midpoint review) validated the core parsing and heuristic logic before more substantial engineering was undertaken.

3.4 Risk Register

Risks were identified at the start of the project and tracked alongside the Kanban board. The current state of the risk register, updated to reflect implementation completion, is shown in Table 2.

Table 2. Project risk register and mitigation status.

ID	Risk	Likelihood	Impact	Mitigation	Status
R1	DNS logs may not contain sufficient data for meaningful behavioural patterns	Medium	High	Early proof-of-concept validated query volume, TLD distribution, and response codes	Mitigated
R2	Selected tools may be unsuitable for small-scale environments	Low	Medium	Lightweight, open-source stack (Pi-hole, Python, Streamlit) validated through dockerised demo	Mitigated
R3	Difficulty interpreting DNS behaviour without misclassifying benign activity	Medium	Medium	Baseline-driven heuristics with explicit thresholds and evidence records	Mitigated
R4	Scope drifting towards security detection or threat classification	Medium	High	Behavioural framing reinforced in code comments, dashboard footer, and report wording	Mitigated
R5	Legal or ethical issues from DNS data collection	Low	High	Metadata-only collection in researcher-owned environment; ethics application submitted	Mitigated
R6	Insufficient time to complete dashboard development	Medium	Medium	Iterative delivery; analysis core delivered before dashboard polish	Mitigated
R7	Limited reproducibility across log formats	Medium	Low	Parser written against dnsmasq format used by Pi-hole; pytest suite locks regex behaviour	Mitigated

3.5 Legal, Ethical, and Professional Issues

3.5.1 Legal considerations

The project collects only DNS metadata: queried domain names, timestamps, query types, and DNS response codes. No payload data, content data, or personally identifiable information is recorded. All DNS logs are generated within a controlled local environment owned and administered by the researcher. Data minimisation, local storage, limited retention, and anonymisation where applicable align the project with data-protection principles of purpose limitation, data minimisation, and storage limitation under the UK GDPR and the Data Protection Act 2018.

The project complies with the Computer Misuse Act 1990 by avoiding any unauthorised access, network scanning, interception of third-party communications, or monitoring of external networks. All testing is conducted exclusively on systems and networks under the direct control of the researcher.

3.5.2 Ethical considerations

DNS metadata can indirectly reveal patterns of system or user behaviour if misused. To mitigate this risk the project performs analysis at an aggregate behavioural level, makes no attempt to profile individual users or devices, and presents results as trends and observations rather than as individual activity logs. The project deliberately avoids interpreting unusual DNS behaviour as malicious, reducing the risk of misrepresentation. The completed ethics form is included in Appendix B.

Two distinct activities were considered. For DNS data analysis, all testing was conducted within a controlled environment managed by the researcher. No external participants provided DNS data and no personal browsing data belonging to others was collected. For dashboard evaluation, three cyber security friends voluntarily reviewed the dashboard with consent. They did not provide or interact with their own DNS logs; instead, they viewed the researcher-generated dashboard outputs and gave informal feedback on clarity, usefulness, and presentation. This kept the evaluation low risk while still providing user-facing feedback on the artefact.

3.5.3 Professional considerations

The project is delivered in line with BCS professional standards. All third-party tools and libraries are used in accordance with their intended purpose and properly attributed. Assumptions and limitations are documented in this report and in the README accompanying the source code. Claims about the system are explicitly behavioural; the dashboard footer states that no security or threat claims are being made, and the heuristic engine wording avoids any framing of findings as detections.

4. Requirements and Specifications

4.1 Intended Users and Personas

Two personas were used to keep design decisions user-focused throughout the project.

Persona 1: Cyber security student. An undergraduate studying networking concepts who wants to understand how DNS behaves under normal conditions, see how traffic patterns change over time, and observe how unusual behaviour emerges without assuming malicious intent. The student finds existing tools focused on blocking or detection, and finds enterprise tools complex and opaque.

Persona 2: Network enthusiast / lab user. An individual operating a small home lab or test network who wants visibility into DNS activity, the ability to experiment with configuration, and interpretable behavioural metrics rather than raw logs.

4.2 User Stories

1. US1: As a user, I want to view DNS query volume over time so that I can understand normal traffic patterns.
2. US2: As a user, I want to see the distribution of DNS response codes so that I can identify unusual error behaviour.
3. US3: As a user, I want to view commonly queried domains and TLDs so that I can understand typical DNS usage.
4. US4: As a user, I want unusual DNS behaviour to be highlighted so that I can investigate deviations from baseline patterns.
5. US5: As a user, I want the system to explain why behaviour is highlighted rather than simply flag it.
6. US6: As a user, I want to compare behaviour across multiple datasets so that I can see how heuristics fire in different conditions.

4.3 User Journey

A typical interaction with the system follows this sequence:

- Start Pi-hole inside Docker using the supplied PowerShell script.
- Generate DNS traffic using one of the scripted profiles, or select an existing controlled dataset.
- Export the resulting Pi-hole/dnsmasq log into the project data folder.
- Run the analysis pipeline to parse the log, calculate metrics, and apply heuristics.
- Open the Streamlit dashboard and choose the active dataset from the sidebar.
- Compare datasets, inspect key metrics, and review any highlighted behavioural observations.
- Interpret the findings as DNS behaviour observations rather than as threat or security claims.

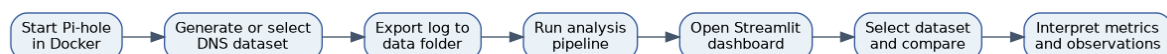


Figure 1. User flow diagram showing the intended interaction sequence.

4.4 Use Case Descriptions

Analyse DNS logs. The user submits a DNS log dataset to the analysis engine, which parses the file, computes baseline metrics, and applies the heuristic suite.

Visualise DNS behaviour. The user opens the Streamlit dashboard, selects a dataset, and inspects the modules to gain insight into normal DNS behaviour.

Identify unusual behaviour. The user inspects highlighted observations on the rule-status and observation modules and traces the evidence to specific domains, time windows, response codes, or TLDs.

Compare across datasets. The user selects multiple datasets in the sidebar and views side-by-side metrics to compare baseline traffic with controlled deviations.

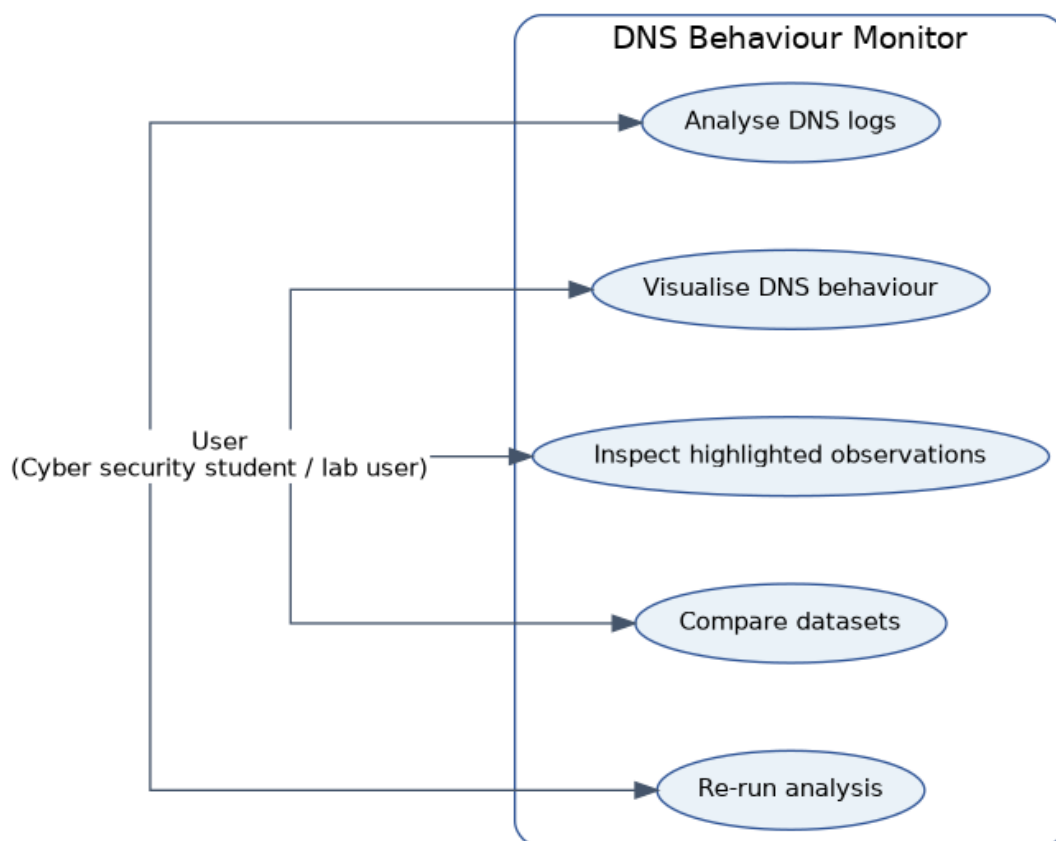


Figure 2. Use case diagram for the DNS Behaviour Monitor.

4.5 Functional Requirements

- FR1: The system shall parse Pi-hole / dnsmasq logs into a normalised schema including timestamp, domain, query type, client, response code, and source.
- FR2: The system shall compute baseline DNS behaviour metrics: query volume over time, top domains, TLD distribution, domain-length distribution, label-count distribution, query-type distribution, and response-code distribution.
- FR3: The system shall identify unusual behaviour using transparent heuristics for long domains, query bursts, NXDOMAIN excess, and rare TLDs.
- FR4: The system shall use baseline metrics to drive heuristic thresholds where a baseline is supplied, and shall fall back to self-derived statistics otherwise.
- FR5: The system shall visualise behaviour through a web-based dashboard with KPI summary, rule status, charts, and detailed observations.
- FR6: The system shall allow comparison of metrics across multiple datasets.

4.6 Non-Functional Requirements

- NFR1: The system shall be lightweight, deployable on a developer laptop, and require no enterprise infrastructure.
- NFR2: The system shall prioritise interpretability over detection accuracy.
- NFR3: The system shall operate only on data generated within a controlled environment.
- NFR4: The system shall comply with legal and ethical constraints described in Section 3.5.
- NFR5: The system shall be reproducible: a single PowerShell command (scripts/demo.ps1) shall start Pi-hole, generate datasets, run analysis, and open the dashboard.
- NFR6: The parser shall achieve a parse success rate of at least 95 percent on query lines.

5. Solution Design

5.1 High-Level Architecture

The system is organised into four logical layers, illustrated in Figure 3. The data source layer contains Pi-hole running inside Docker and the controlled datasets collected or generated for evaluation. The ingestion layer parses dnsmasq-format logs and converts them into a consistent internal schema. The analysis layer derives behavioural features, computes baseline metrics, and applies heuristic rules. The output and presentation layer writes CSV/JSON files and renders the results through a Streamlit dashboard.

This separation of responsibilities supports maintainability. For example, the dashboard can be redesigned without changing the parser, and future log formats can be added without rewriting the metrics or heuristic engines.

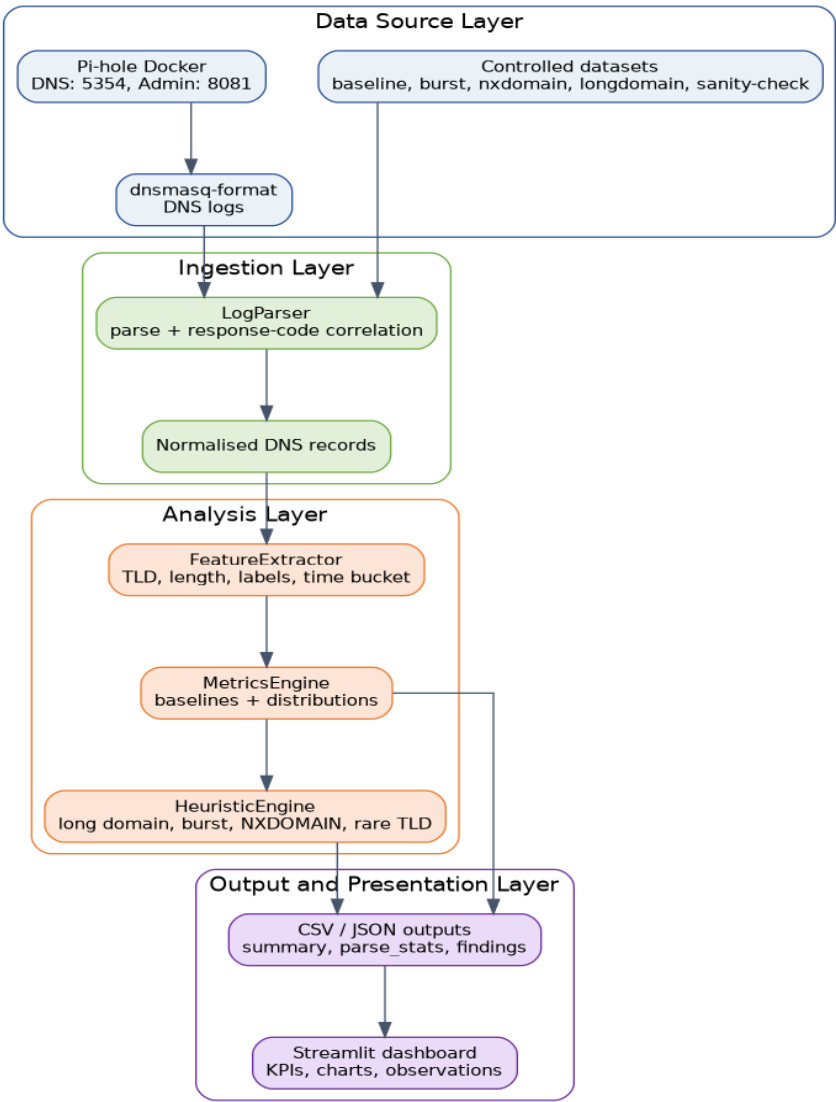


Figure 3. High-level system architecture of the DNS Behaviour Monitor.

5.2 Data Flow and Processing Pipeline

Figure 4 provides a clearer view of the processing pipeline. A raw DNS log is parsed into query records, response codes are correlated from reply lines, records are normalised, and derived features are added. Metrics are then calculated, the baseline summary is loaded where available, and heuristic rules produce evidence-bearing observations. The outputs are saved as CSV and JSON files before being loaded by the dashboard for interpretation.

Each stage can be checked independently. This was important during implementation because parser behaviour, feature extraction, metrics, heuristic output, and dashboard rendering could be tested separately.

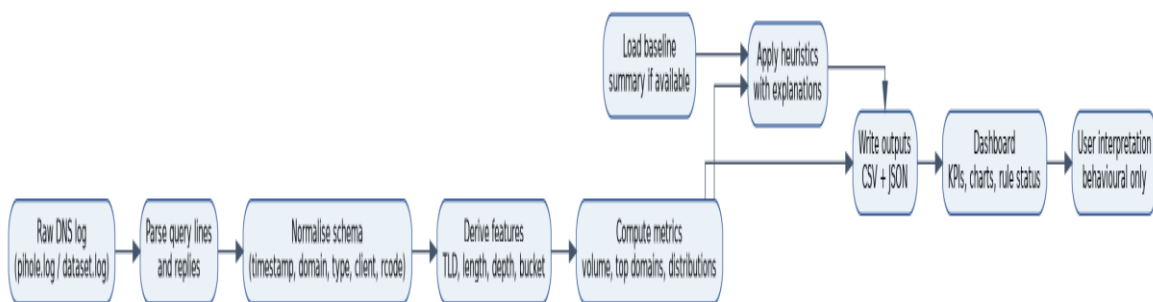


Figure 4. Data flow diagram showing the DNS behaviour analysis pipeline.

5.3 Data Model and Derived Features

Each DNS query is represented internally using a normalised schema with the fields timestamp, domain, query type, client identifier, response code, and source. Client identifiers are not used for profiling and are only retained where required for parser consistency. From the normalised schema the feature extractor derives top-level domain, domain length, label count, and a time bucket. These derived features support structural, temporal, and distributional analysis without packet inspection or payload analysis.

Although the implemented system stores analysis outputs using CSV and JSON rather than a relational database, an ERD was created to document the logical data structure handled by the tool. The model shows how log sources, DNS records, analysis runs, baseline metrics, heuristic rules, heuristic findings, and output files relate to each other.

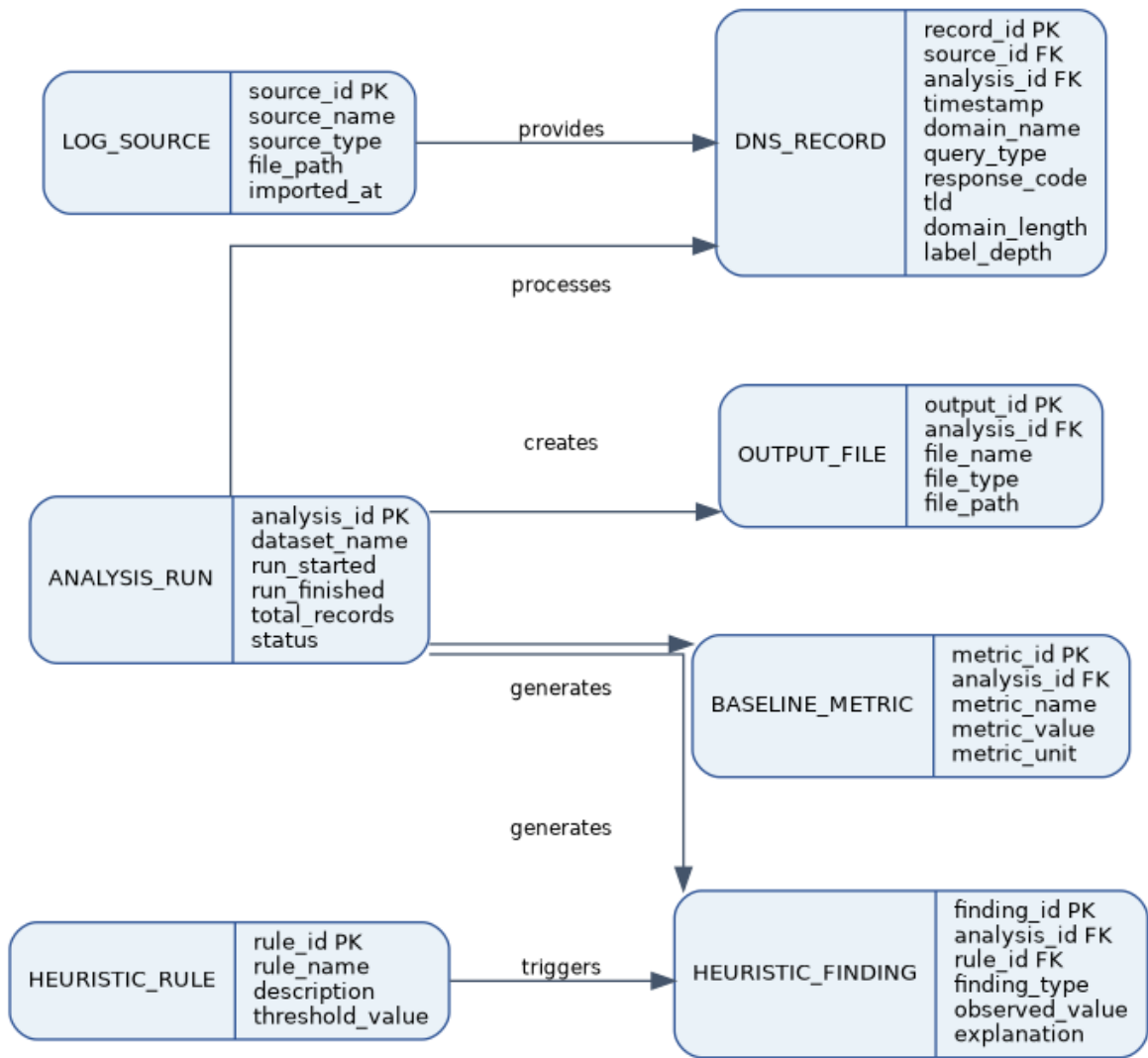


Figure 5. Entity relationship diagram for the DNS Behaviour Monitor logical data model.

5.4 Core Modules and Responsibilities

The system is decomposed into five core Python modules under `src/`. The `LogParser` detects log structure, extracts query fields, and correlates response codes from following reply, cached, config, or Pi-hole hostname lines. The `FeatureExtractor` adds TLD, length, label-count, and time-bucket features. The `MetricsEngine` computes baseline tables and writes summary JSON and CSV files. The `HeuristicEngine` applies the four rule-based checks and writes findings JSON. The `Streamlit` dashboard loads those outputs and renders the multi-module interface.

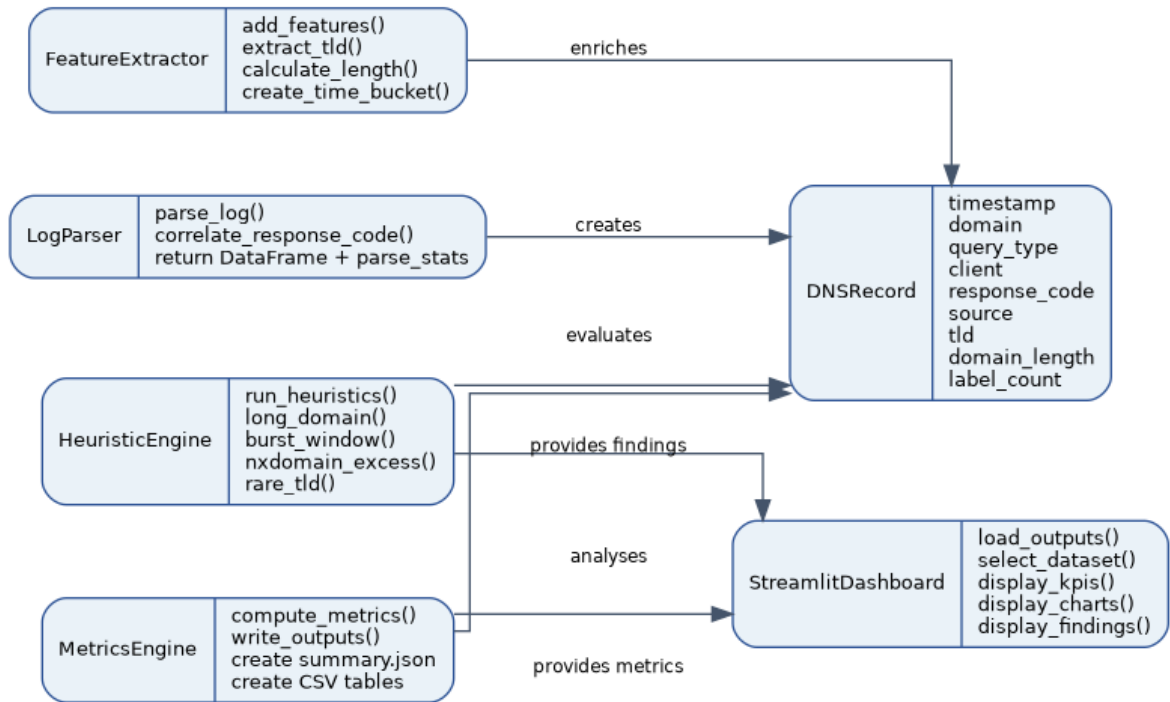


Figure 6. UML class diagram showing the main software components of the DNS Behaviour Monitor.

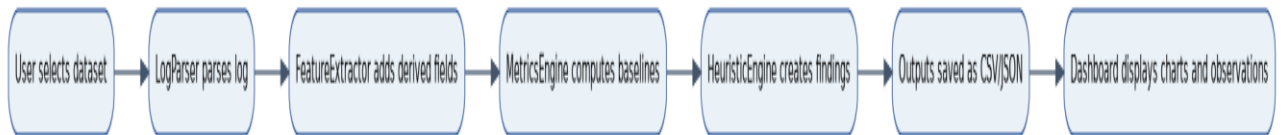


Figure 7. UML sequence diagram for the DNS log analysis workflow.

5.5 Heuristic Design

Four heuristics were defined to align with the project's analytical scope:

Table 3. Heuristic rules and threshold logic.

Rule	Condition	Threshold logic
long_domain	Domain length is far above the baseline maximum	max(baseline mean + 2 standard deviations, baseline max). Hard fallback at 60 chars when no baseline is supplied.
burst_window	Time-bucket query count is far above the baseline 95th percentile	baseline p95 × 2 (10-second buckets); falls back to self-p95 × 2 if no baseline.
nxdomain_excess	NXDOMAIN ratio is far above baseline ratio	Ratio ≥ 20 percent AND ≥ 20 percentage points above baseline.
rare_tld	TLD frequency is below 1 percent of total queries	Fixed 1 percent fraction; behavioural rather than baseline-relative.

The thresholds were chosen to produce explainable, evidence-bearing findings while minimising false positives on benign baseline traffic. Each heuristic emits a finding record containing a type, an evidence dictionary, and a `why_flagged` sentence so that observations remain interpretable to the user.

5.6 Design Constraints and Assumptions

- All DNS data is generated within a controlled environment.
- Data volumes are modest and suitable for in-memory processing with pandas.
- Behavioural interpretation prioritises clarity over exhaustive detection.
- The system is not intended for real-time enterprise deployment.

6. Solution Implementation

6.1 Implementation Overview

This section describes the implementation as delivered, organised by sprint. The final repository contains a Docker/Pi-hole data source, PowerShell automation, a Python analysis pipeline, generated/collected datasets, CSV/JSON outputs, parser tests, and a Streamlit dashboard. The submitted code also includes a supplementary sanity-check dataset and corresponding output files, which were used as extra dashboard evidence during final review.

Implementation evidence is referenced throughout this section and supported by screenshots in Appendix G. The screenshots show the dashboard running locally at localhost:8505 with the baseline, sanity-check, burst, long-domain, and NXDOMAIN datasets.

6.2 Sprint 1: Containerised Data Source

The first sprint established a reproducible DNS data source. Pi-hole was deployed using Docker Compose with DNS exposed on host port 5354 and the admin interface on host port 8081. This avoided conflicts with the Windows DNS Client service on port 53. The Docker configuration and PowerShell helper scripts allow the environment to be started and stopped consistently without manually configuring Pi-hole each time.

6.3 Sprint 2: Traffic Generation and Dataset Export

The second sprint implemented scripted dataset generation and export. The Python traffic generator uses dnspython to send DNS queries to 127.0.0.1:5354. Profiles were created for baseline traffic, burst traffic, NXDOMAIN-heavy traffic, and long-domain traffic. Export scripts copy the resulting Pi-hole log into the data folder. The submitted repository also contains a sanity-check log used to confirm that the dashboard can load an additional dataset and highlight long-domain observations in a mixed log.

6.4 Sprint 3: Log Parser and Schema

The parser in src/log_parser.py reads dnsmasq-format logs line by line. Query lines contain the timestamp, query type, domain, and client, while response codes are recovered by correlating each query with the following reply, cached, config, or Pi-hole hostname line for the same domain. The parser writes a normalised DataFrame and parse statistics, including query attempts, parsed lines, skipped lines, parse-success rate, replies seen, and queries with response codes.

6.5 Sprint 4: Feature Extraction and Metrics

The feature extractor adds derived fields including TLD, domain length, label count, and time bucket. The metrics engine calculates query volume over time, top domains, TLD distribution, domain-length distribution, label-count distribution, query-type distribution, and response-code distribution. These are saved as summary.json and CSV files under outputs/<dataset>/, making the results inspectable without running the dashboard.

6.6 Sprint 5: Heuristic Engine

The heuristic engine implements four transparent rules: burst_window, nxdomain_excess, long_domain, and rare_tld. The rules remain deliberately behavioural and do not label activity as malicious. Each finding contains a type, an evidence dictionary, and a why_flagged explanation. This design supports the project aim of helping users understand DNS behaviour rather than building a threat-detection product.

6.7 Sprint 6: Streamlit Dashboard

The dashboard in app/streamlit_app.py presents the analysis through a dark-themed Streamlit interface. It includes a dataset selector, comparison selector, re-run analysis button, KPI cards, heuristic-status cards, query-volume chart, top-domain list, and additional modules for distributions and observations. Figure 8 shows the dashboard displaying the baseline dataset with no observations flagged. Additional dashboard evidence for the other datasets is included in Appendix G.

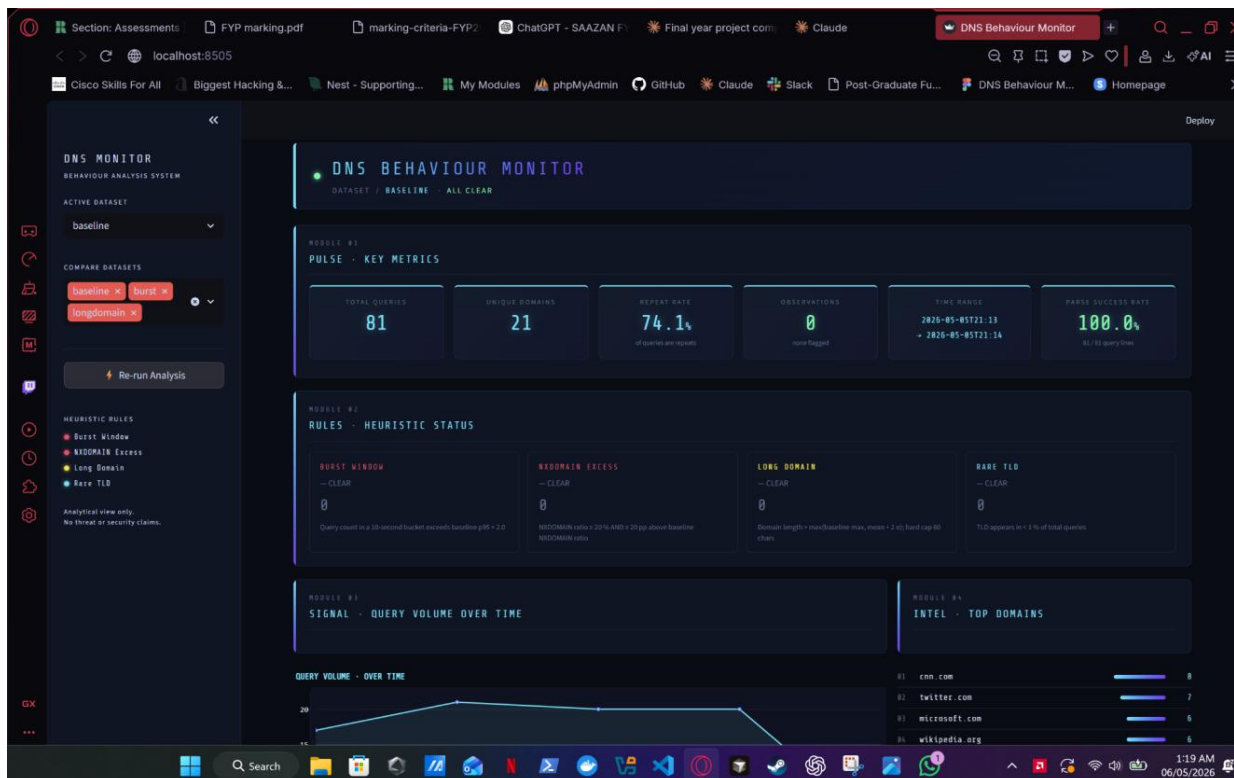


Figure 8. Streamlit dashboard KPI and heuristic-status view.

6.8 Sprint 7: Automation and Test Harness

The final sprint added automation and test support. `demo.ps1` starts Pi-hole, waits until DNS responds on port 5354, generates the four main datasets, runs analysis, and opens the dashboard. `run_evaluation.ps1` is provided to regenerate the controlled datasets, analyse them against the baseline, and check expected heuristic firings. The pytest suite under `tests/` contains six parser tests covering schema columns, timestamp parsing, parse-success rate, regex matching, response-code correlation, and the Pi-hole self-resolved hostname case. The final verification run completed with all six tests passing.

7. Solution Testing and Evaluation

7.1 Evaluation Objectives

The evaluation was designed to address the following objectives:

- Verify that the parser reliably processes Pi-hole/dnsmasq query lines and meets the 95 percent success-rate target.
- Check whether the dashboard displays baseline, sanity-check, burst, long-domain, and NXDOMAIN datasets correctly.
- Assess whether heuristic observations match the behaviour intentionally generated in each dataset.
- Confirm that the baseline dataset remains clear, reducing evidence of false positives in normal controlled traffic.
- Collect informal feedback from cyber security peers on whether the dashboard is visually clear and useful for understanding DNS behaviour.
- Identify limitations in the datasets, heuristic interpretation, and user-facing evaluation.

7.2 Methodology

The evaluation combined three forms of evidence. First, automated parser tests were executed using pytest. Second, dashboard evidence screenshots were captured for the baseline, sanity-check, burst, long-domain, and NXDOMAIN datasets. Third, informal peer feedback was collected from three cyber security friends who reviewed the dashboard and commented on presentation and usefulness. The screenshots used as evidence are included in Appendix G.

7.3 Technical Evaluation: Parser and Pipeline

The pytest suite under tests/test_parser.py contains six tests. These tests check that the parser produces the expected schema, parses timestamps correctly, meets the parser-success target, matches query and reply regular expressions correctly, correlates response codes, and handles the Pi-hole self-resolved hostname case. The final verification run completed successfully with all six tests passing.

The dashboard screenshots also show a 100 percent parse-success rate for each dataset. Table 4 summarises the values visible in the evidence screenshots. The values are based on query lines rather than total raw log lines because dnsmasq logs also contain reply, cached, configuration, and service messages that are expected to be skipped by the parser.

Table 4. Parse-success evidence.

Dataset	Query lines parsed	Parse success rate	Dashboard evidence
baseline	81 / 81	100.0%	Figure 14
sanity-check	165 / 165	100.0%	Figure 15
burst	101 / 101	100.0%	Figure 16
longdomain	41 / 41	100.0%	Figure 17
nxdomain	50 / 50	100.0%	Figure 18

7.4 Dataset-Based Behavioural Evaluation

Five datasets were reviewed in the final dashboard evidence. The baseline dataset represents normal controlled DNS behaviour and remained clear. The sanity-check dataset contains a realistic mixed log and produced four long-domain observations. The burst dataset produced 101 queries and triggered both the `burst_window` and `rare_tld` rules. The long-domain dataset produced 41 unique long domains and triggered `long_domain` and `nxdomain_excess` observations. The NXDOMAIN dataset produced 50 unique failed lookup domains and also triggered `nxdomain_excess` and `long_domain` observations, because the generated non-existent domains were structurally long.

Table 5 summarises the heuristic outcomes shown in the dashboard evidence. The co-firing of `long_domain` and `nxdomain_excess` on the NXDOMAIN and long-domain datasets is treated as an interpretive limitation rather than a defect: the generated test inputs intentionally create domains that are both long and non-resolving.

Table 5. Heuristic observations across evaluation datasets.

Dataset	Queries	Observations	Triggered rules	Interpretation
baseline	81	0	None	Normal baseline remained clear
sanity-check	165	4	<code>long_domain</code> (4)	Mixed log highlighted long-domain behaviour
burst	101	2	<code>burst_window</code> (1); <code>rare_tld</code> (1)	High-volume burst plus one rare-TLD observation
longdomain	41	21	<code>long_domain</code> (20); <code>nxdomain_excess</code> (1)	Generated long domains also returned NXDOMAIN
nxdomain	50	21	<code>nxdomain_excess</code> (1); <code>long_domain</code> (20)	Failed lookups used long generated domain names

Screenshots supporting these values are provided in Appendix G as Figures 14 to 18.

7.5 Dashboard Usability Evaluation

A formal think-aloud study was not completed. Instead, three cyber security friends voluntarily reviewed the running dashboard with consent. They did not provide or interact with their own DNS data; they only viewed the researcher-generated dashboard outputs and gave informal feedback.

The participants generally found the dashboard visually pleasing and considered the information useful for understanding DNS behaviour at a high level. However, the feedback was limited and not detailed enough to claim strong usability validation. This experience showed that more structured user evaluation would be needed in future work. In particular, a task-based evaluation or questionnaire would provide stronger evidence than informal comments alone.

7.6 Evaluation Metrics Summary

Table 6. Evaluation metrics, methods, and outcomes.

Area	Metric	Method	Target	Outcome
Log parsing	Parse success rate	pytest + dashboard parse value	>= 95%	100% on evidence screenshots
Baseline false positives	Findings on baseline	Baseline dashboard evidence	Zero	Zero observations flagged
Behaviour sensitivity	Rule triggering on controlled datasets	Dashboard rule-status cards	Expected rule(s) trigger	Burst, long-domain, NXDOMAIN, and sanity-check datasets triggered relevant observations
Dataset comparison	Multiple datasets loaded	Dashboard selector and screenshots	Visible comparison support	Baseline, burst, longdomain, nxdomain, and sanity-check were loaded
Interpretability	Informal peer feedback	Three cyber security friends reviewed dashboard	Useful formative feedback	Dashboard described as visually pleasing and useful, but feedback depth was limited

7.7 Limitations of the Evaluation

Four limitations apply to the evaluation. First, the datasets were generated or collected in a controlled local environment and may not generalise to enterprise networks or larger user populations. Second, some heuristics co-fired because generated datasets can contain multiple unusual structural properties at once, such as long domains that also return

NXDOMAIN. Third, the sanity-check dataset demonstrated that real or mixed logs can include local resolver behaviour such as pi.hole and reverse lookup domains, which requires careful contextual interpretation. Fourth, the dashboard evaluation was informal and involved only three cyber security friends, so it provides useful formative feedback but not strong summative usability evidence.

8. Discussion

This section returns to the research question: how DNS behaviour can be characterised and visualised in a controlled environment to distinguish normal usage from unusual activity without relying on security-focused detection. The evidence in Sections 6 and 7 shows that the project answers this question through a transparent pipeline that parses DNS logs, derives behavioural metrics, applies baseline-informed heuristics, and presents the results in an interpretable dashboard.

The baseline dataset remained clear with zero observations, which supports the decision to avoid aggressive fixed thresholds. The burst dataset produced 101 queries and triggered the `burst_window` rule, showing that sudden high-volume DNS activity can be highlighted relative to baseline behaviour. The burst screenshot also showed one rare-TLD observation, demonstrating that the TLD rule can fire when a TLD appears below the configured one-percent threshold. The long-domain and NXDOMAIN datasets both produced 21 observations, including expected `long_domain` and `nxdomain_excess` findings.

The evaluation also shows why the project must remain behavioural rather than security-detection focused. The NXDOMAIN and long-domain datasets co-fired because generated test domains were both structurally long and non-resolving. This does not mean that the activity is malicious; it means that the log contains more than one unusual structural property. This finding supports the project framing that DNS observations require context and should not be overinterpreted.

Compared with the related tools discussed in Section 2.3, the DNS Behaviour Monitor occupies a lightweight middle ground. Pi-hole provides useful operational counts but does not explain behavioural baselines. Cloudflare and DNS-OARC provide valuable large-scale insight but are not designed for small controlled environments. The artefact developed here is smaller in scope, but it is transparent, reproducible, and suitable for educational experimentation.

The informal dashboard evaluation was useful but limited. Three cyber security friends found the interface visually pleasing and useful for understanding the outputs, but the feedback was not detailed enough to prove strong usability. This is an important finding in itself: a visually polished dashboard still needs structured user testing before claims about usability can be made confidently.

Overall, the project demonstrates that DNS behaviour can be made understandable through simple metrics, visualisation, and explainable heuristics. Its main contribution is not advanced detection accuracy, but a transparent educational artefact that makes DNS behaviour visible and encourages careful interpretation.

9. Conclusion

9.1 Summary

This project set out to characterise and visualise DNS behaviour in a controlled environment without relying on security-focused detection. The final artefact combines a Docker/Pi-hole data source, scripted traffic generation, a Python parsing and analysis pipeline, baseline metrics, four transparent heuristics, CSV/JSON outputs, automated tests, and a Streamlit dashboard.

The evaluation evidence showed 100 percent parse success on the dashboard evidence screenshots for baseline, sanity-check, burst, long-domain, and NXDOMAIN datasets. The baseline remained clear, while the other datasets triggered observations consistent with their generated behaviour. The dashboard was reviewed informally by three cyber security friends, who found it visually pleasing and useful, although the evaluation remained limited. The project therefore meets its main aim as a lightweight DNS behaviour monitoring and learning tool, while also identifying realistic areas for future improvement.

9.2 Recommendations for Further Work

Several extensions are realistic within a follow-up project of similar scale:

- Enrich the rare-TLD evaluation by introducing a deliberate rare-TLD dataset profile, for example generated queries to uncommon or new TLDs, so that the rule can be tested more systematically.
- Disambiguate co-firing rules by adding a primary-cause field to each finding. This would help explain whether a finding is primarily caused by domain length, response-code ratio, burst volume, or TLD rarity.
- Strengthen usability evidence with a larger participant pool and a structured questionnaire such as a System Usability Scale survey. This would turn the current informal feedback into stronger evaluation evidence.
- Add a bring-your-own-data flow to the dashboard that accepts an uploaded log file, runs analysis, and presents the same modules. This would broaden educational use while keeping the project transparent and controlled.

9.3 Reflection

Looking back across the project, three reflections stand out from a personal perspective and are written here in the first person, in line with the report template.

I learned the practical value of separating concerns at the architectural level. Early in the project I was tempted to fold parsing, feature extraction, and heuristics into a single script because the data volumes were modest. Separating the parser, feature extractor, metrics engine, heuristic engine, and dashboard made the system easier to test and easier to explain in the report.

I also learned how much harder it is to write about a project than to build it. By the midpoint review I had a working proof-of-concept but had under-described the design choices that

produced it, particularly around heuristic thresholds. Supervisor feedback helped me explain why each threshold was chosen rather than only describing what the threshold was.

Where I fell short, the main issue was evaluation depth. I was able to show the dashboard to three cyber security friends and they found it visually pleasing and useful, but I did not collect detailed task-based feedback. If I were starting again, I would plan the usability evaluation earlier and use a short structured questionnaire. I would also design the datasets so that each heuristic is exercised separately before combining behaviours.

On balance, the project taught me that lightweight, transparent tools have real value in cyber security education. They make assumptions visible, which is important for learning. The project also strengthened my understanding of DNS, Python data processing, Streamlit dashboards, ethical handling of network metadata, and professional reporting.

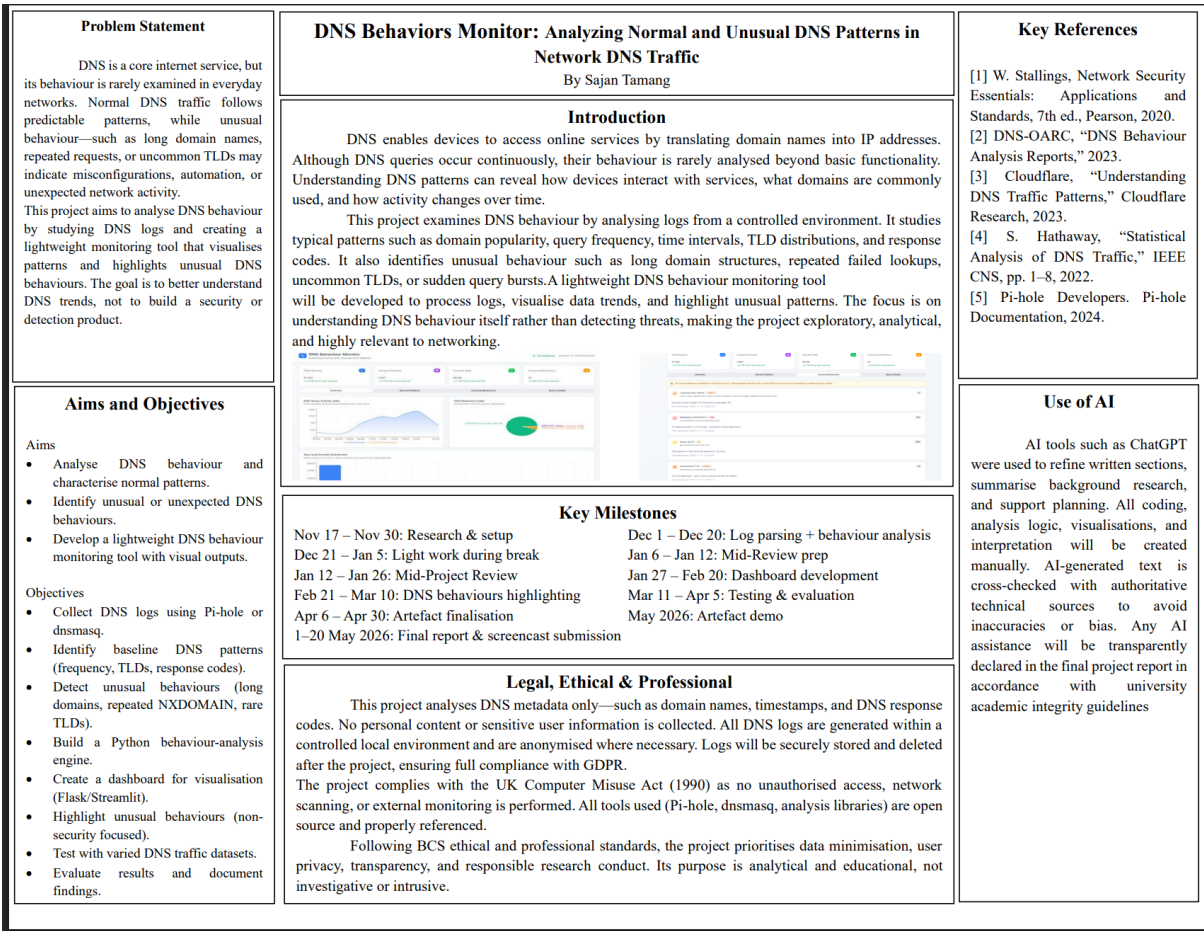
10. References

- [1] P. V. Mockapetris, "Domain names — concepts and facilities," RFC 1034, Nov. 1987. [Online]. Available: <https://www.rfc-editor.org/info/rfc1034>. [Accessed: Apr. 2026].
- [2] P. V. Mockapetris, "Domain names — implementation and specification," RFC 1035, Nov. 1987. [Online]. Available: <https://www.rfc-editor.org/info/rfc1035>. [Accessed: Apr. 2026].
- [3] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, "DNS performance and the effectiveness of caching," *IEEE/ACM Transactions on Networking*, vol. 10, no. 5, pp. 589–603, Oct. 2002, doi: 10.1109/TNET.2002.803905.
- [4] M. Crovella and B. Krishnamurthy, *Internet Measurement: Infrastructure, Traffic and Applications*. Hoboken, NJ, USA: Wiley, 2006.
- [5] DNS Operations, Analysis, and Research Center (DNS-OARC), "DNS measurement reports," 2023. [Online]. Available: <https://www.dns-oarc.net>. [Accessed: Jan. 2026].
- [6] Cloudflare, "Understanding DNS traffic patterns," Cloudflare Research, 2023. [Online]. Available: <https://blog.cloudflare.com>. [Accessed: Jan. 2026].
- [7] Pi-hole Developers, "Pi-hole documentation," 2024. [Online]. Available: <https://docs.pi-hole.net>. [Accessed: Jan. 2026].
- [8] W. Stallings, *Network Security Essentials: Applications and Standards*, 7th ed. Harlow, UK: Pearson, 2020.
- [9] British Computer Society, "BCS code of conduct for members," 2022. [Online]. Available: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>. [Accessed: Apr. 2026].
- [10] Information Commissioner's Office, "Guide to the UK General Data Protection Regulation (UK GDPR)," 2024. [Online]. Available: <https://ico.org.uk>. [Accessed: Apr. 2026].
- [11] Streamlit Inc., "Streamlit documentation," 2024. [Online]. Available: <https://docs.streamlit.io>. [Accessed: Apr. 2026].
- [12] Plotly Technologies Inc., "Plotly Python graphing library," 2024. [Online]. Available: <https://plotly.com/python>. [Accessed: Apr. 2026].
- [13] B. Sanderson, dnspython, version 2.4. [Online]. Available: <https://www.dnspython.org>. [Accessed: Mar. 2026].

11. Appendices

Appendix A — Project Proposal Poster

Figure 9. Project proposal poster.



Appendix B — Ethics Form

Figure 10. Ethics form evidence, page 1.

Id	57
Start time	03/02/2026 8:17
Completion time	03/02/2026 9:13
Email	tamangs2@roehampton.ac.uk
Name	Sajan Tamang
Your name	SAJAN TAMANG
Your student ID	Z22614816
Your course	CYBER SECURITY
Your module	Undergraduate Final Year Project (CMP-X301-0)
Year of study	2026
Todays date	03/02/2026
Project title	DNS Behaviour Monitor
Project description	The motivation for this project is to improve understanding of Domain Name System (DNS) behaviour in small and controlled network environments. DNS is a core Internet service, yet its behaviour is rarely examined beyond basic operational monitoring or security-focused detection. This project aims to explore DNS traffic as a source of behavioural insight rather than as an indicator of malicious activity. The research will involve the analysis of DNS log data generated within a controlled local environment owned and administered by the researcher. DNS metadata such as queried domain names, timestamps, and DNS response codes will be analysed to characterise normal DNS behaviour and highlight unusual patterns using baseline-driven and heuristic-based methods. No packet payloads, personal data, or user-identifiable information will be collected or analysed. The artefact produced will be a lightweight DNS behaviour monitoring tool implemented using Python. The tool will parse DNS logs, compute behavioural metrics, and present results through interpretable outputs

Figure 11. Ethics form evidence, page 2.

	designed to support exploratory analysis and learning rather than security detection or enforcement.
Project start and end date	
Will your research involve human subjects?	Yes
What form of research best describes your plans?	Usability testing
How will you recruit participants?	Participants will be recruited through convenience sampling from the researcher's peers studying on BSc computing and cyber security programmes at the University of Roehampton. Recruitment will be conducted via informal student communication channels (e.g. group chats). Participation will be voluntary, anonymous, and participants may withdraw at any time.
Will your research be done ONLY with Roehampton University staff and students, or with external participants?	ONLY Roehampton
Declaration (1)	I can confirm that all Information supplied on this form is correct and my work complies with ethical standards; I will not be recording any 'special category' information about individuals; I will not be carrying out any research directly with under 18 year olds or vulnerable adults; If any activities may require a risk assessment I will speak with my Supervisor and prepare this.; All data collected, including audio and video recordings, will be deleted when I cease to be a Roehampton University student; If I make any video or audio recordings, these will be held securely, not used for any purpose other than this research; I will hold data obtained for research only

Figure 12. Ethics form evidence, page 3.

	on Roehampton University's OneDrive; I will anonymise all data collected from individuals in my report; The data collected will only be used for the purpose of this study; All questionnaires or surveys will include participant information and a consent option; If my research involves people outside Roehampton University I will ensure that I have obtained written consent for their participation and confirmation of their right to withdraw.; If Roehampton students or staff participate in my research, I will ensure that I have explained the project carefully, obtained their verbal consent and explained that they may withdraw from the study at any time.
Declaration (2)	
Supervisor name	Badis Aoun
Supervisor signature to confirm that the project meets ethics guidelines	
This research requires ethical approval from the Module leader	No
Module leader signature	

Appendix C — GitHub Repository and Project Management

<https://github.com/SajanTamangX/dns-behaviour-monitor.git>

Figure 13. GitHub repository and project management evidence.

The figure consists of two screenshots. The top screenshot shows the GitHub repository 'dns-behaviour-monitor' by SajanTamangX. It displays the repository structure with files and folders like 'app', 'data', 'fyp files', 'outputs', 'pihole', 'scripts', 'src', 'tests', '.gitignore', 'CHANGELOG.md', and 'LICENSE'. The bottom screenshot shows the project management board '@SajanTamangX's FYP' in a Kanban view. It lists 12 tasks with their status, size, and priority.

Title	Status	Size	Priority
1 DNS traffic generator with 4 profiles	Done	M	P0
2 Dataset export + validation pipeline	Done	S	P0
3 Dockerized Pi-hole environment (ports 5354/8081)	Done	M	P0
4 Evaluation automation script	Done	M	P0
5 Heuristic engine (4 baseline-driven rules)	Done	L	P0
6 Metrics engine + CSV/JSON outputs	Done	M	P0
7 Parser test suite	Done	M	P0
8 Parser: normalized schema + response-code correlation	Done	L	P0
9 Release published: v1.0.0	Done	XS	P0
10 Streamlit dashboard (multi-module analytics view)	Done	XL	P0
11 Submission documentation + release artefacts	Ready	M	P0
12 Feature extraction stage	Done	S	P1

Appendix D — Risk Register

ID	Risk	Likelihood	Impact	Mitigation	Status
R1	DNS logs may not contain sufficient data for meaningful behavioural patterns	Medium	High	Early proof-of-concept validated query volume, TLD distribution, and response codes	Mitigated
R2	Selected tools may be unsuitable for small-scale environments	Low	Medium	Lightweight, open-source stack (Pi-hole, Python, Streamlit) validated through dockerised demo	Mitigated
R3	Difficulty interpreting DNS behaviour without misclassifying benign activity	Medium	Medium	Baseline-driven heuristics with explicit thresholds and evidence records	Mitigated
R4	Scope drifting towards security detection or threat classification	Medium	High	Behavioural framing reinforced in code comments, dashboard footer, and report wording	Mitigated
R5	Legal or ethical issues from DNS data collection	Low	High	Metadata-only collection in researcher-owned environment; ethics application submitted	Mitigated
R6	Insufficient time to complete dashboard development	Medium	Medium	Iterative delivery; analysis core delivered before dashboard polish	Mitigated
R7	Limited reproducibility across log formats	Medium	Low	Parser written against dnsmasq format used by Pi-hole; pytest suite locks regex behaviour	Mitigated

Appendix E — Supervisor Communications and Mid-point Review Feedback

Meeting 1 — Project Selection and Initial Guidance. Project topic discussed and confirmed as DNS behaviour analysis. Supervisor explained assessment structure and marking criteria. Guidance provided on project proposal poster requirements and on appropriate scope for a BSc cyber security project.

Meeting 2 — Poster Review and Formative Feedback. Poster reviewed and feedback provided. Evaluation recommended to focus on user understanding of DNS behaviour, not only technical correctness. Use of a Kanban board suggested to manage backlog, milestones, and progress tracking. Literature review advised to be more summarised and analytical, avoiding excessive description. Project aims recommended to be clarified and strengthened.

Meeting 3 — Midterm Review. Supervisor advised listing, analysing, and summarising key academic papers; competitor DNS tools to be explicitly listed and compared; and the legal, ethical and professional considerations section to be developed in depth. All three points have been addressed in this final report.

Appendix F — Use of Generative AI

Generative AI tools were used during the project to refine the readability of written sections drafted by the author, brainstorm report structure, and improve the final organisation of the document. All code, analysis logic, dataset generation, evaluation results, and interpretation remain the author's own work. AI-assisted text was checked against authoritative technical sources and personal understanding before inclusion. This declaration is consistent with the use-of-AI statement on the project proposal poster.

Appendix G — Testing and Dashboard Evidence

This appendix contains the dashboard screenshots used as evidence for the final evaluation tables in Section 7. The screenshots show the active dataset, total query count, parse-success rate, observation count, and heuristic status cards for each evaluated dataset.

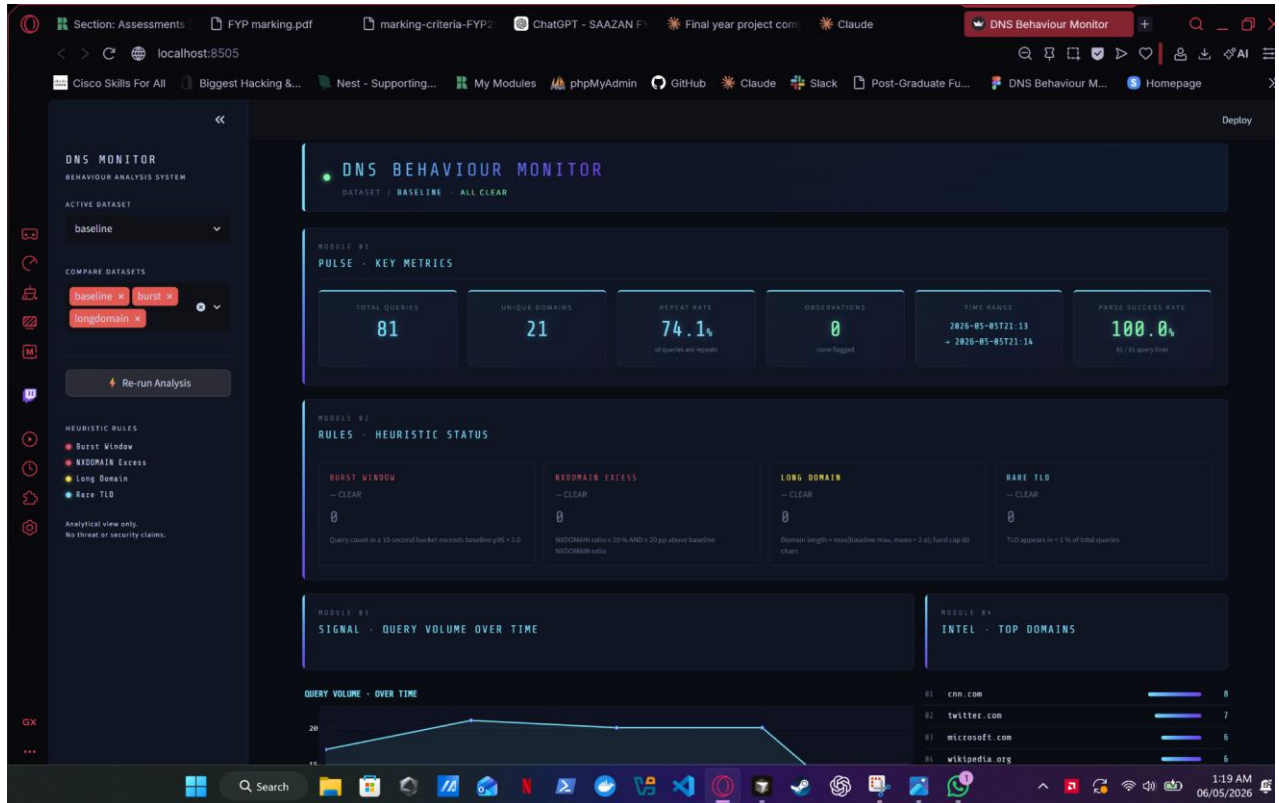


Figure 14. Baseline dashboard evidence showing 81 queries, zero observations, and 100 percent parse success.

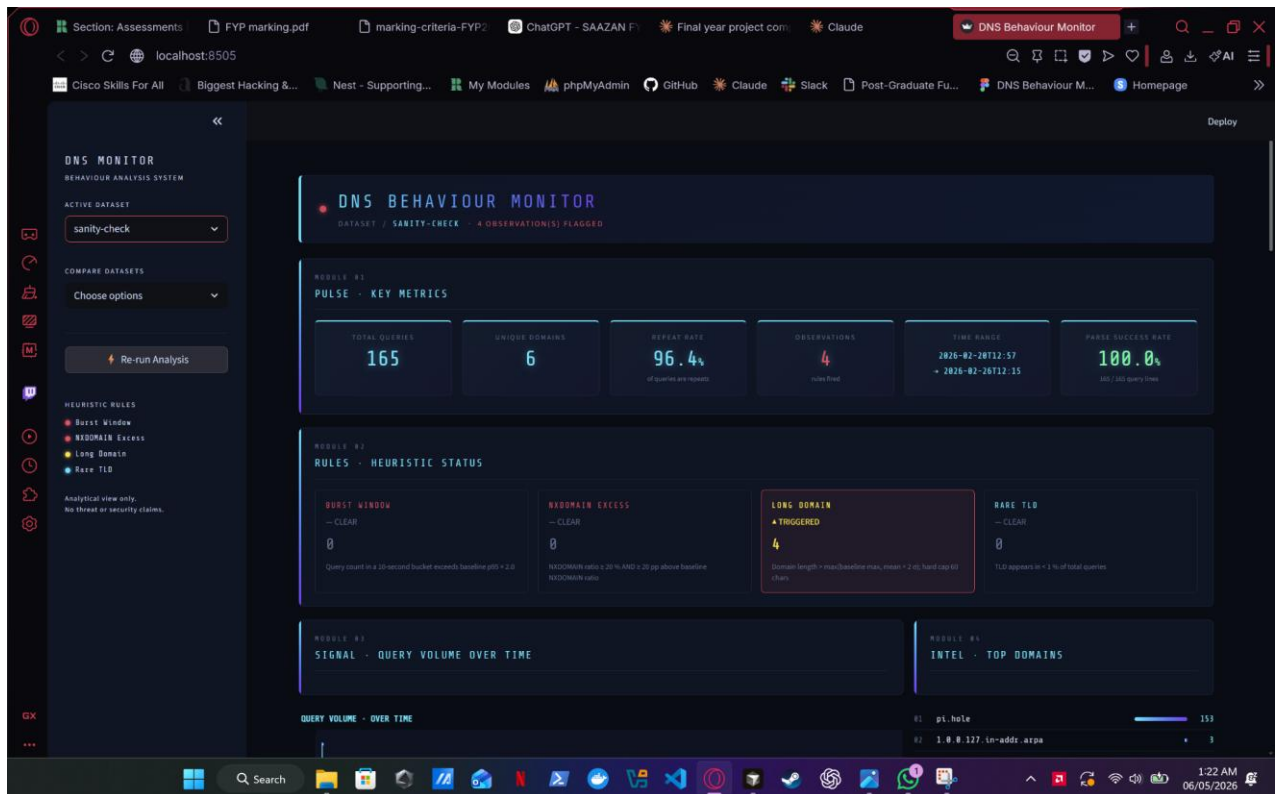


Figure 15. Sanity-check dashboard evidence showing 165 queries, four long-domain observations, and 100 percent parse success.

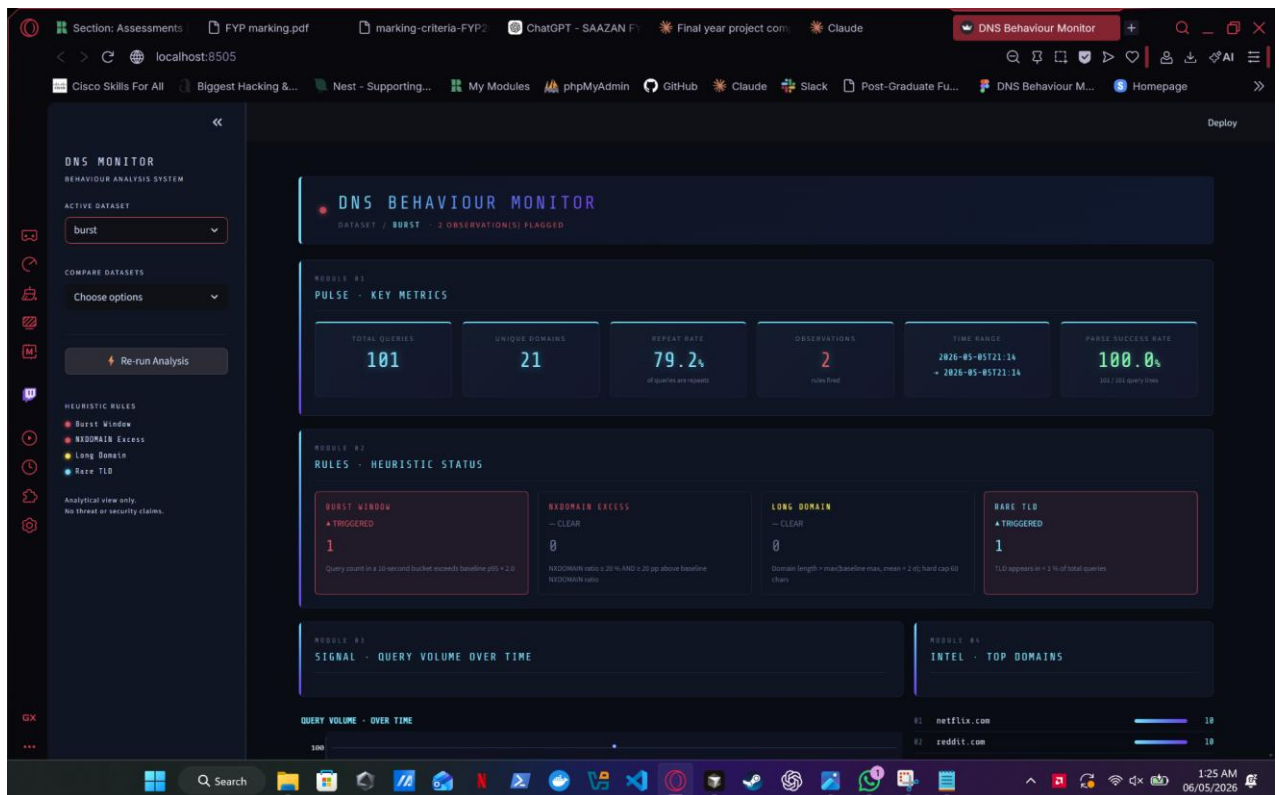


Figure 16. Burst dashboard evidence showing 101 queries, two observations, and 100 percent parse success.

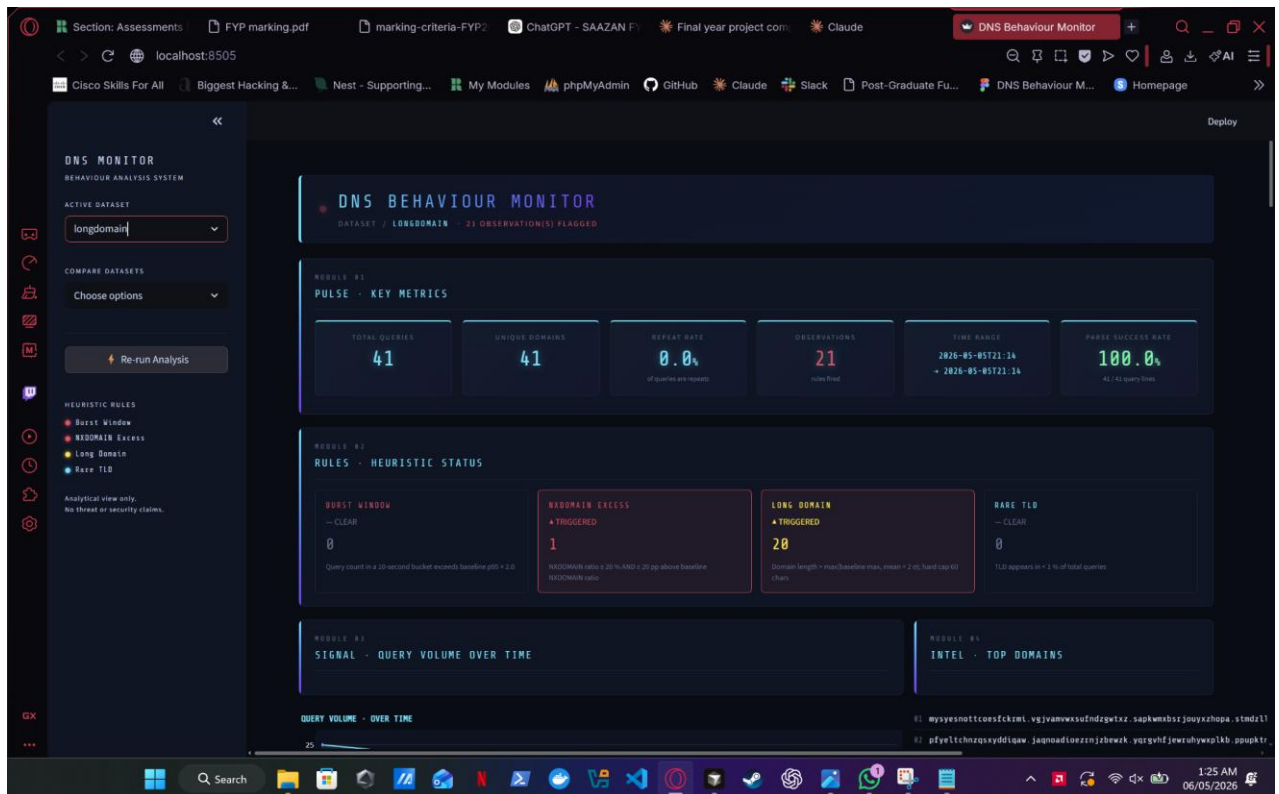


Figure 17. Long-domain dashboard evidence showing 41 queries, 21 observations, and 100 percent parse success.

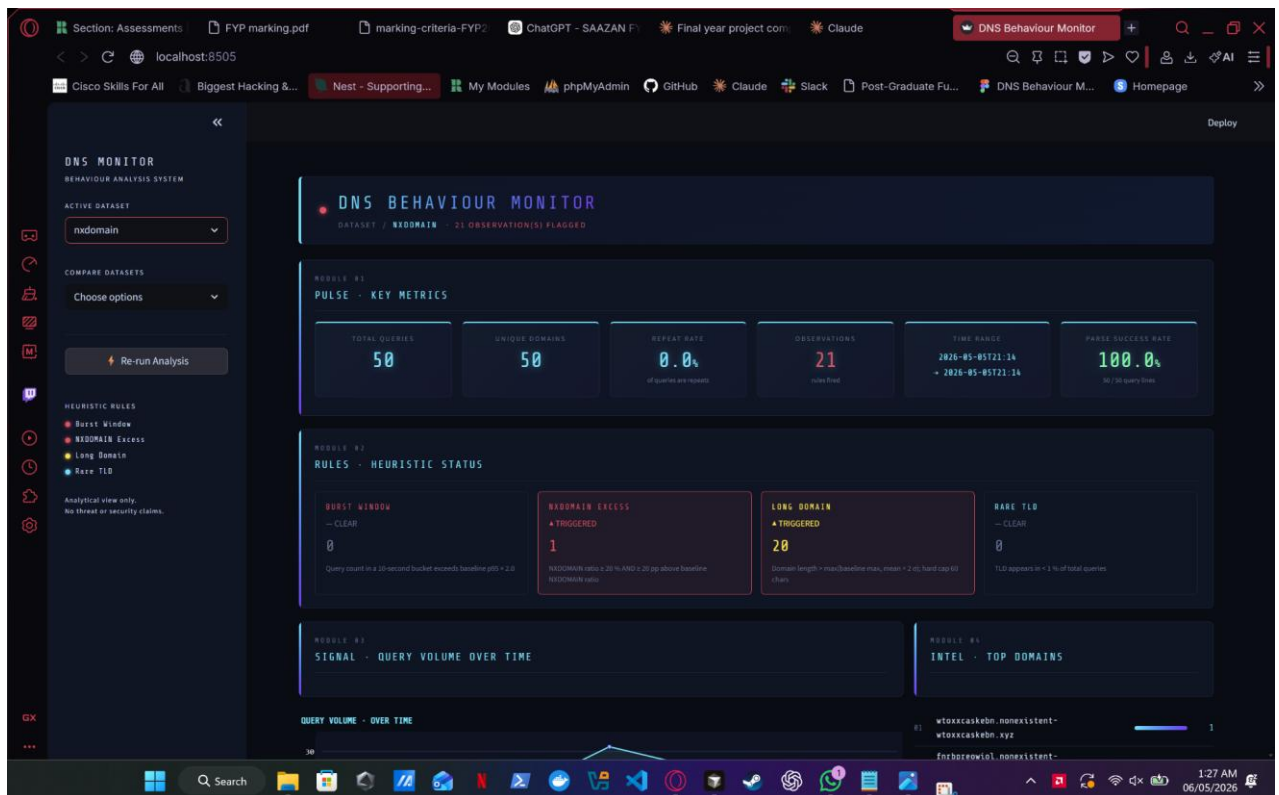


Figure 18. NXDOMAIN dashboard evidence showing 50 queries, 21 observations, and 100 percent parse success.